

Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Facultad de Ingeniería
Departamento de Ingeniería de Sistemas e Industrial
Estructura de Datos (2026-2)

Repositorio GitHub: https://github.com/jpiratobat/ED_Listas_Pilas_Colas.git

Jorge Eduardo Piratoba Tocarruncho
jpiratobat@unal.edu.co

1. Objetivo

El presente informe tiene como objetivo abordar el diseño, la implementación y el análisis de complejidad de las estructuras de datos lineales fundamentales: **Listas Enlazadas** (*Singly LinkedList*), **Listas Doblemente Enlazadas** (*Doubly LinkedList*), **Pilas** (*DynamicArrayStack*) y **Colas** (*DynamicArrayQueue*). Estas estructuras conforman la base teórica y práctica para el desarrollo de sistemas de software eficiente, optimización del uso de memoria y ejecución de algoritmos complejos.

En este laboratorio se analizan las operaciones de inserción, eliminación y búsqueda, evaluando su comportamiento dinámico mediante el uso de anotaciones Big- O .

2. Implementación

2.1. Interfaz Lista (MyList)

Define la estructura contractual e interfaz genérica que establece las operaciones fundamentales para la gestión de datos lineales de tipo secuencial.

- `pushFront(T data)`: Inserta un nuevo elemento al principio de la lista.
- `pushBack(T data)`: Inserta un nuevo elemento al final de la lista.
- `topFront()`: Devuelve el primer elemento de la lista sin eliminarlo.
- `popFront()`: Extrae y devuelve el primer elemento de la lista.
- `topBack()`: Devuelve el último elemento de la lista sin eliminarlo.
- `popBack()`: Extrae y devuelve el último elemento de la lista.
- `find(T data)`: Devuelve la referencia al nodo que contiene la primera aparición del dato indicado, o `null` si el dato no se encuentra en la lista.
- `erase(T data)`: Elimina todas las apariciones del dato indicado en la lista.
- `addBefore(Node target, T data)`: Inserta un nuevo elemento (`data`) inmediatamente antes de la primera aparición del elemento indicado (`target`).

- `addAfter(Node target, T data)`: Inserta un nuevo elemento (`data`) inmediatamente después de la primera aparición del elemento indicado (`target`).
- `isEmpty()`: Determina si la lista no contiene elementos.
- `getSize()`: Devuelve la cantidad de elementos almacenados en la lista.

Esta interfaz se implementa en cuatro variantes, resultado de combinar el tipo de enlace (simple o doble) con la presencia o ausencia de un puntero `tail`: `SinglyLinkedList`, `SinglyLinkedListTail`, `DoublyLinkedList` y `DoublyLinkedListTail`.

2.2. Lista Simplemente Enlazada (`SinglyLinkedList` y `SinglyLinkedListTail`)

Una lista simplemente enlazada es una estructura de datos lineal formada por una secuencia de nodos conectados de forma unidireccional mediante referencias. A diferencia de los arreglos estáticos, no requiere un bloque de memoria contiguo.

- **Nodo**: Estructura básica que almacena el elemento (`data`) y una referencia (`next`) al siguiente nodo.
- **Cabeza (Head)**: Puntero de entrada al primer nodo de la lista.
- **Cola (Tail)**: Puntero directo al último nodo de la lista, optimizando las inserciones al final a $O(1)$.

2.3. Lista Doblemente Enlazada (`DoublyLinkedList` y `DoublyLinkedListTail`)

A diferencia de la lista simplemente enlazada, los nodos en esta estructura cuentan con punteros bidireccionales. Aunque ofrece mayor flexibilidad de recorrido (hacia adelante y hacia atrás), incrementa el consumo de memoria al almacenar una referencia adicional por nodo.

- **Nodo Doble**: Contiene el dato almacenado y dos referencias: `next` (siguiente nodo) y `prev` (nodo anterior).
- **Cabeza (Head)**: Puntero al primer nodo de la lista.
- **Cola (Tail)**: Puntero al último nodo de la lista.

2.4. Pila (`MyStack`)

Estructura de datos de acceso restringido que opera bajo la filosofía LIFO (*Last In, First Out*), donde el último elemento en ingresar es el primero en ser extraído.

- `push(T data)`: Agrega un elemento a la parte superior (*top*) de la pila.
- `pop()`: Elimina y retorna el elemento en la parte superior.
- `peek()`: Consulta el elemento en la parte superior sin retirarlo.
- `isEmpty()`: Retorna un valor booleano que indica si la pila está vacía.
- `getSize()`: Retorna la cantidad total de elementos contenidos en la pila.
- `delete(T data)`: Elimina las apariciones del valor especificado.

2.5. Pila sobre Arreglo Dinámico (DynamicArrayStack)

Implementación de la pila sobre un arreglo de capacidad variable. El elemento en la cima corresponde a la última posición ocupada del arreglo, de modo que **push** y **pop** operan sobre ese extremo sin desplazar elementos. Cuando la pila alcanza su capacidad máxima al ejecutar un **push**, el arreglo redimensiona dinámicamente su tamaño, duplicando su capacidad actual. Esto implica la creación de un nuevo arreglo en memoria y la copia secuencial de todos los elementos preexistentes.

Estrategia de crecimiento y análisis. Se eligió el crecimiento por duplicación de la capacidad. Cada redimensionamiento cuesta $O(n)$, pero ocurre pocas veces: si la capacidad inicial es c y se realizan n inserciones, las copias se producen cuando el arreglo tiene tamaño $c, 2c, 4c, \dots$, y el total de elementos copiados es

$$c + 2c + 4c + \dots + \frac{n}{2} < n.$$

2.6. Cola (MyQueue)

Estructura de datos basada en el principio FIFO (*First In, First Out*), donde los elementos se insertan por un extremo (*rear*) y se extraen por el extremo opuesto (*head*).

- **enqueue(T data):** Añade un nuevo elemento al final de la cola.
- **dequeue():** Retira y retorna el primer elemento de la cola.
- **front():** Consulta el primer elemento de la cola sin eliminarlo.
- **isEmpty():** Retorna un valor booleano indicando si la cola se encuentra vacía.
- **getSize():** Retorna el número actual de elementos en la cola.
- **delete(T data):** Elimina las apariciones del dato especificado dentro de la cola.

2.7. Cola sobre Arreglo Dinámico (DynamicArrayQueue)

Esta implementación utiliza un “arreglo circular” para evitar mayor consumo en memoria de elementos tras cada operación **dequeue**. Se mantiene un índice **front** (posición del primer elemento) y un contador **size**; la posición de inserción de **enqueue** se calcula como

$$\text{rear} = (\text{rear} + 1) \text{ mód } \text{capacity},$$

y **dequeue** solo avanza **front** de forma circular, por lo que ambas operaciones no requieren desplazar elementos y se ejecutan en $O(1)$.

Cuando el arreglo alcanza su capacidad máxima, se asigna un nuevo contenedor de mayor tamaño (duplicando la capacidad, con el mismo análisis amortizado de la pila), se reordenan secuencialmente los elementos partiendo desde el índice **front** hacia el nuevo arreglo y se reajustan los punteros principales (**front** pasa a ser 0).

3. Análisis de Complejidades

En los Cuadros 1, 2 y 3 se presenta el resumen de complejidad teórica, en notación Big- O , para cada estructura de datos analizada. En los apartados siguientes se justifica el costo de cada método y se formula la hipótesis que se contrastará con las mediciones experimentales.

3.1. Supuestos y notación

- n denota el número de elementos almacenados en la estructura al momento de ejecutar la operación.
- Las operaciones **find**, **erase** y **delete** reciben un dato y lo localizan por valor, es decir, mediante un recorrido de la estructura. En cambio, **addBefore** y **addAfter** reciben la referencia al nodo objetivo (obtenida previamente con **find**), por lo que su costo no incluye la búsqueda.
- Las listas mantienen un contador de elementos, por lo que **getSize()** no requiere recorrido. **isEmpty()** se resuelve comparando **head** con **null** (o el contador con 0).
- En las estructuras sobre arreglo dinámico se distingue entre el **peor caso** de una operación individual y el **caso amortizado** sobre una secuencia de operaciones. La capacidad crece por duplicación (Secciones 2.5 y 2.7).

3.2. Complejidad teórica en MyList

Cuadro 1: Complejidad de métodos en MyList

Método	Singly (sin Tail)	Singly (con Tail)	Doubly (sin Tail)	Doubly (con Tail)
pushFront(data)	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack(data)	$O(n)$	$O(1)$	$O(n)$	$O(1)$
topFront()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popFront()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
topBack()	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popBack()	$O(n)$	$O(n)$	$O(n)$	$O(1)$
find(data)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
isEmpty()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
getSize()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
erase(data)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
addBefore(...)	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter(...)	$O(1)$	$O(1)$	$O(1)$	$O(1)$

La justificación de cada grupo de métodos es la siguiente:

- **pushFront, topFront, popFront, isEmpty, getSize** – $O(1)$ en las cuatro variantes. Solo involucran el puntero **head** y el contador de elementos: se crea o desengancha un nodo y se actualizan un número constante de referencias, sin importar n . En la lista doble, **popFront** solo agrega anular el **prev** del nuevo primer nodo.
- **pushBack y topBack**. Sin **tail** es necesario recorrer los n nodos hasta llegar al último, con costo $O(n)$, tanto en la lista simple como en la doble. Con **tail** el último nodo es accesible directamente, con costo $O(1)$.
- **popBack**. Requiere conocer el penúltimo nodo para actualizar el final de la lista. En la lista simple no existe enlace hacia atrás, por lo que se debe recorrer la lista aun contando con **tail**: $O(n)$ en ambas variantes. En la doble sin **tail** también se debe recorrer para llegar al último nodo: $O(n)$. Solo la doble con **tail** obtiene el penúltimo nodo mediante **tail.prev**, con costo $O(1)$.

- **find.** En el peor caso el dato buscado es el último elemento o no está en la lista, por lo que se recorren los n nodos: $O(n)$ en todas las variantes.
- **erase.** Al eliminar todas las apariciones del dato, la lista debe recorrerse completa aunque ya se haya encontrado una coincidencia. Cada eliminación individual sobre un nodo ya localizado es $O(1)$ (en la lista simple, gracias a que se conserva el nodo anterior durante el recorrido; en la doble, mediante **prev** y **next**), por lo que el costo total está dominado por el recorrido: $O(n)$ en todas las variantes.
- **addAfter** – $O(1)$ en las cuatro variantes. Al recibir la referencia al nodo objetivo, basta enlazar el nuevo nodo a continuación de este (**next**, y **prev** en la lista doble), un número constante de asignaciones. Cuando el objetivo es el último nodo, las variantes con **tail** actualizan además ese puntero en tiempo constante.
- **addBefore.** En la lista doble, el nodo anterior al objetivo se obtiene directamente mediante **target.prev**, por lo que la inserción es $O(1)$ con o sin **tail**. En la lista simple no existe enlace hacia atrás, así que es necesario recorrer la lista desde **head** hasta encontrar el nodo que precede al objetivo, con costo $O(n)$ en ambas variantes.

3.3. Complejidad teórica en MyStack y MyQueue

Cuadro 2: Complejidad de métodos en MyStack

Método	Peor Caso	Caso Amortizado
push(data)	$O(n)$	$O(1)$
pop()	$O(1)$	$O(1)$
peek()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$
getSize()	$O(1)$	$O(1)$
delete(data)	$O(n)$	$O(n)$

Cuadro 3: Complejidad de métodos en MyQueue

Método	Peor Caso	Caso Amortizado
enqueue(data)	$O(n)$	$O(1)$
dequeue()	$O(1)$	$O(1)$
front()	$O(1)$	$O(1)$
isEmpty()	$O(1)$	$O(1)$
getSize()	$O(1)$	$O(1)$
delete(data)	$O(n)$	$O(n)$

- **push y enqueue.** Si hay capacidad disponible, se escribe en la posición correspondiente en $O(1)$. Cuando el arreglo está lleno se crea un arreglo de capacidad doble y se copian los n elementos, lo que da un peor caso de $O(n)$. Como este redimensionamiento ocurre solo cuando el tamaño alcanza potencias sucesivas de la capacidad inicial, el costo total de n inserciones es $O(n)$ y el costo amortizado por inserción es $O(1)$ (ver Sección 2.5).

- **pop**, **peek**, **dequeue** y **front**. Acceden directamente a la posición de la cima (o de **front** en la cola circular) y actualizan un índice, con costo $O(1)$ sin importar n . Se asume que no se reduce la capacidad del arreglo al extraer elementos.
- **isEmpty** y **getSize**. Consultan el contador de elementos: $O(1)$.
- **delete**. Debe recorrer los n elementos para localizar las apariciones del valor y compactar el arreglo desplazando los elementos posteriores. Este costo es $O(n)$ tanto en el peor caso como en el amortizado, ya que no depende del redimensionamiento.

3.4. Hipótesis

La hipótesis de este análisis es que **el comportamiento experimental de cada método coincidirá con su complejidad teórica** presentada en los Cuadros 1, 2 y 3, para cada una de las implementaciones. En concreto, se espera que:

- Los métodos $O(1)$ (por ejemplo **pushFront**, **popFront**, **addAfter**, **isEmpty**, **getSize**, **peek**, **front**, **pop** y **dequeue**) presenten un tiempo prácticamente independiente del tamaño de la estructura, sin diferencias apreciables entre las distintas variantes de lista.
- Los métodos $O(n)$ (por ejemplo **find**, **erase**, **delete** y **addBefore** en las listas simples) presenten un tiempo que crece proporcionalmente a n .
- El puntero **tail** reduzca **pushBack** y **topBack** de $O(n)$ a $O(1)$ en las listas simple y doble, y que únicamente **DoublyLinkedListTail** reduzca **popBack** a $O(1)$.
- El enlace **prev** de las listas dobles reduzca **addBefore** de $O(n)$ a $O(1)$, tanto con **tail** como sin él.
- **push** y **enqueue** sobre arreglo dinámico exhiban un costo amortizado constante, con picos ocasionales en los instantes en que el arreglo se redimensiona.

3.5. Metodología de medición

Para cada tamaño $N \in \{10, 100, 1000, 10000, 100000\}$ se ejecutaron 5 iteraciones. En cada una, el método **exec** cronometra N llamadas consecutivas del mismo método con **Instant.now()** y reporta el tiempo acumulado en microsegundos (μs), obtenido de **Duration.toNanos()/1000**. Se eligieron μs porque en nanosegundos las cifras acumuladas resultan demasiado grandes para compararse de forma intuitiva, y en milisegundos las operaciones $O(1)$ quedarían reducidas a valores casi nulos.

- Los datos de **find**, **erase** y **delete** son aleatorios (**Random**), y las pruebas se ejecutan en secuencia sobre las mismas estructuras.
- Los gráficos usan escala logarítmica; cuando el tiempo es $0 \mu s$ no es posible graficarlo.

4. Resultados

4.1. Estructuras sobre Listas

Los gráficos y las tablas del Anexo son consistentes con lo esperado: los métodos que requieren recorrer la lista presentan un crecimiento lineal, mientras que los demás mantienen una complejidad constante. Se confirma la complejidad con la teoría.

- **Impacto del puntero Tail:** En las Figuras 1 y 3, la ausencia de tail provoca un crecimiento lineal $O(n)$ en las operaciones al final (**pushBack** y **topBack**), dado que se debe recorrer la lista nodo por nodo. En contraste, las Figuras 2 y 4 muestran un tiempo constante $O(1)$ para dichas operaciones.
- **Diferencia en popBack:** Únicamente la estructura **DoublyLinkedListTail** (Figura 4) logra ejecutar **popBack** en tiempo constante $O(1)$, debido a que los enlaces hacia atrás (**prev**) permiten desenganchar el último nodo sin recorrer toda la lista. En las demás variantes, esta operación conserva un crecimiento lineal $O(n)$.
- **Búsqueda y eliminación:** Métodos como **find** y **erase** presentan un crecimiento lineal $O(n)$ en las cuatro variantes, pues ni el puntero **tail** ni el doble enlace evitan el recorrido de la lista.
- **Operaciones en el inicio y consultas de estado:** **pushFront**, **popFront**, **isEmpty** y **getSize** mantienen un tiempo constante $O(1)$, sin diferencias relevantes entre las cuatro implementaciones.

4.2. Estructuras sobre Arreglos Dinámicos

Las Figuras 6 y 5 representan el rendimiento y el costo amortizado de las implementaciones basadas en arreglos redimensionables. Tanto los gráficos como las tablas del Anexo son consistentes con la teoría.

- **push y enqueue:** Mantienen un tiempo prácticamente constante, coherente con su costo amortizado $O(1)$, a pesar de que su peor caso es $O(n)$ cuando el arreglo alcanza su capacidad máxima y se redimensiona.
- **pop, peek, dequeue, front, isEmpty y getSize:** Presentan una complejidad constante $O(1)$, independiente del número de elementos almacenados.
- **delete:** Es el único método que exhibe un crecimiento lineal $O(n)$, ya que debe recorrer el arreglo para localizar el valor y compactar los elementos restantes.

4.3. Análisis del Costo Amortizado

Aunque una inserción individual cuesta $O(n)$ cuando el arreglo se llena, el costo por operación sobre una secuencia es constante. Esto se demuestra con el **método del banquero** (contable): se cobra un costo amortizado de **3 unidades** por cada inserción y se lleva una cuenta de créditos.

- Una unidad paga la escritura del elemento en el arreglo.
- Las 2 unidades restantes se guardan como crédito sobre ese elemento.
- Cuando el arreglo de capacidad c está lleno y se duplica, copiar los c elementos cuesta c unidades. Tras el redimensionamiento hay c elementos y espacio para c más; las siguientes c inserciones acumulan $2c$ unidades de crédito, suficientes para pagar la copia de los $2c$ elementos en el siguiente redimensionamiento.

Como el crédito nunca es negativo, el costo real total de n inserciones es a lo sumo $3n$, es decir, $O(1)$ amortizado por inserción, a pesar del peor caso $O(n)$. El mismo argumento aplica a **enqueue** en la cola circular. El tiempo acumulado no permite ver cada redimensionamiento por separado, pero el total medido (10^5 inserciones en unos milisegundos) es consistente con este análisis.

4.4. Limitaciones de la medición

- **Resolución del temporizador:** `Instant.now()` usa el reloj del sistema. Muchos tiempos de métodos $O(1)$ son 0 y los no nulos se agrupan cerca de múltiplos de $\approx 200 \mu s$ (≈ 1 ms entre 5 iteraciones), lo que indica una resolución efectiva cercana a 1 ms. Para esos métodos solo puede afirmarse que su costo es despreciable frente al de los $O(n)$.

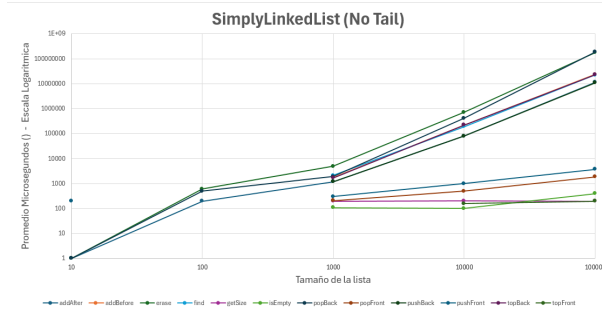


Figura 1: Rendimiento de *SinglyLinkedList* (Sin Tail).

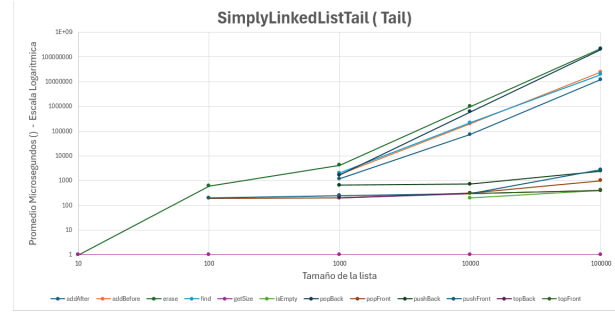


Figura 2: Rendimiento de *SinglyLinkedListTail* (Con Tail).

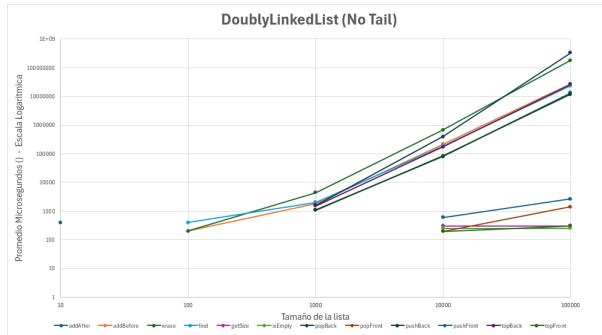


Figura 3: Rendimiento de *DoublyLinkedList* (Sin Tail).

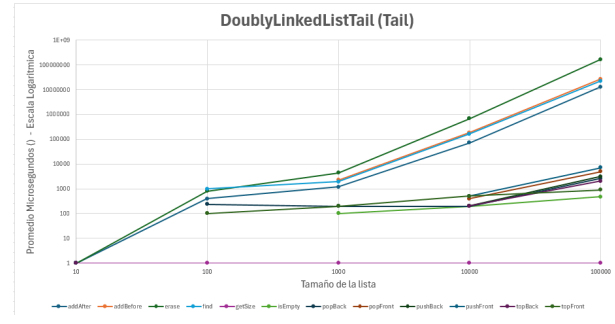


Figura 4: Rendimiento de *DoublyLinkedListTail* (Con Tail).

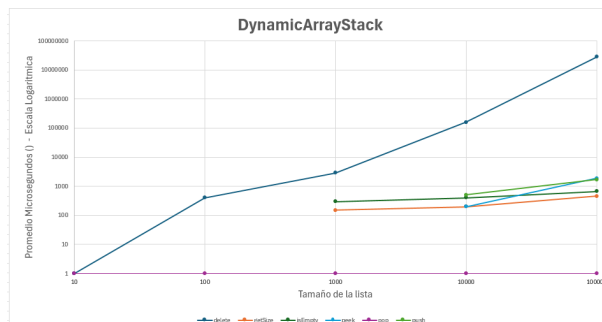


Figura 5: Tiempos de ejecución en *DynamicArrayStack* (push vs pop).

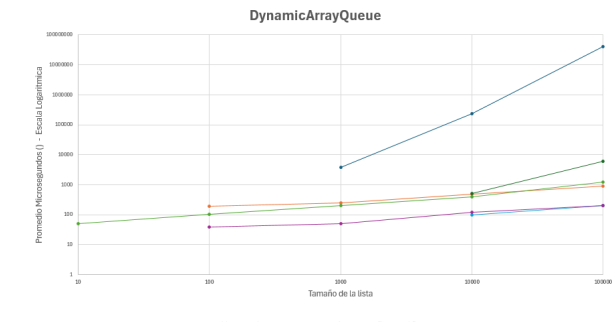


Figura 6: Tiempos de ejecución en *DynamicArrayQueue* (enqueue vs dequeue).

5. Relación entre MyList, MyQueue, MyStack

Las estructuras de datos lineales restringidas como las Pilas (*MyStack*) y las Colas (*MyQueue*) pueden interpretarse como abstracciones especializadas de una Lista (*MyList*). A continuación, se realiza la comparación entre los métodos equivalentes de estas estructuras, con la que cuenta menor complejidad.

1. Inserción en Pila (push) vs. pushFront en Lista:

- **Implementación:** *SinglyLinkedList* (Simple sin *Tail*).
- **Justificación:** Insertar al inicio de una lista enlazada simple solo requiere actualizar el puntero *head* y la referencia *next* del nuevo nodo en tiempo constante $\mathcal{O}(1)$. No se requiere un puntero *tail* ni dobles enlaces (*prev*), lo que minimiza la sobrecarga de memoria por nodo.

2. Inserción en Cola (enqueue) vs. pushBack en Lista:

- **Implementación :** *SinglyLinkedListTail* (Simple con *Tail*).
- **Justificación:** Al contar con un puntero directo al último nodo (*tail*), la inserción al final se realiza en $\mathcal{O}(1)$ sin necesidad de recorrer la lista completa en $\mathcal{O}(n)$. Mantener una estructura simplemente enlazada es suficiente, evitando el espacio adicional que requeriría una lista doblemente enlazada.

3. Extracción en Pila (pop) y Cola (dequeue) vs. popFront / popBack:

- **Extracción en Pila (pop):** Se equipara con **popFront** en *SinglyLinkedList*, logrando un desenganche directo en $\mathcal{O}(1)$.
- **Extracción en Cola (dequeue):** Se equipara con **popFront** en *SinglyLinkedListTail*, removiendo el elemento del frente en $\mathcal{O}(1)$ mientras se conserva la referencia del final.

4. Eliminación Arbitraria (delete vs. erase):

- **Implementación :** *DoublyLinkedListTail* (Doble con *Tail*).
- **Justificación:** Si se conoce la referencia al nodo, una lista doblemente enlazada permite reenlazar los punteros *prev* y *next* de forma inmediata en $\mathcal{O}(1)$. No obstante, en la búsqueda del valor a eliminar, todas las variantes de lista requieren un recorrido lineal de peor caso $\mathcal{O}(n)$.

6. Conclusiones

6.1. Criterios de Selección y Condiciones de Uso

La elección de la estructura de datos lineal óptima depende directamente del patrón de acceso a los datos y de las restricciones de recursos del sistema:

- ***SinglyLinkedList* (Sin *Tail*):** Es la opción más liviana cuando la aplicación realiza lecturas e inserciones exclusivamente al inicio (**pushFront**/**popFront**). Su uso es desaconsejado si se requieren inserciones o accesos frecuentes al final de la secuencia, ya que degeneran en $\mathcal{O}(n)$.

- ***SinglyLinkedListTail (Con Tail)***: Es la estructura idónea para la implementación de colas basadas en nodos, permitiendo inserciones al final (**pushBack**) y extracciones al inicio (**popFront**) en tiempo estricto $\mathcal{O}(1)$ sin la sobrecarga de un doble enlace.
- ***DoublyLinkedListTail (Doble con Tail)***: Se debe seleccionar cuando se requieren recorridos bidireccionales o la eliminación arbitraria de nodos (**erase**) en medio de la estructura en tiempo $\mathcal{O}(1)$ (teniendo la referencia directa al nodo), o bien para operaciones de extracción en el extremo final (**popBack**) en $\mathcal{O}(1)$.
- ***DynamicArrayStack y DynamicArrayQueue***: Son la elección preferente cuando el volumen de datos es predecible o cuando se requiere minimizar la fragmentación de memoria y aprovechar la localidad de referencia. La cola basada en arreglo circular previene la degradación por desplazamiento de índices, ofreciendo un rendimiento altamente eficiente.

6.2. Escenarios de Aplicación Real de Pilas y Colas

- **Casos de uso de Pilas (*LIFO*)**:
 - **Gestión de la pila de llamadas (*Call Stack*)**: Control de la ejecución de métodos, alcance de variables locales y llamadas recursivas en lenguajes de programación.
 - **Mecanismos de Deshacer/Rehacer (*Undo/Redo*)**: Procesamiento de estados anteriores en editores de texto y software de diseño gráfico.
- **Casos de uso de Colas (*FIFO*)**:
 - **Planificación de tareas (*CPU Scheduling*)**: Algoritmos de ordenamiento de procesos en el sistema operativo (p. ej., *Round Robin*).
 - **Recorridos en Grafos**: Algoritmo de búsqueda en anchura (*BFS*) e implementación de ordenamientos topológicos (algoritmo de Kahn) para la resolución de dependencias.

6.3. Arreglos Dinámicos vs. Listas Enlazadas en Java

Los arreglos dinámicos se destacan principalmente por su bajo consumo de memoria auxiliar. Dado que almacenan los datos de forma contigua en memoria permiten un acceso aleatorio indexado en tiempo constante $\mathcal{O}(1)$ sin requerir almacenamiento adicional por elemento. Sin embargo, su principal desventaja radica en la latencia por redimensionamiento: cuando la capacidad del arreglo se agota, la operación de inserción cae a un peor caso de $\mathcal{O}(n)$ debido a la reasignación de memoria y al copiado completo de los elementos, además de poder generar cierta memoria inutilizada debido al factor de crecimiento.

Por otro lado, las listas enlazadas ofrecen como ventaja principal una asignación de memoria dinámicamente estricta y un rendimiento predecible, garantizando que las inserciones y eliminaciones en los extremos se ejecuten en tiempo $\mathcal{O}(1)$ sin sufrir jamás los picos de redimensionamiento. Su mayor desventaja en la Máquina Virtual de Java (JVM) se encuentra en la alta sobrecarga de memoria y la constante presión sobre el Recolector de Basura (*Garbage Collector*) debido a la rápida asignación y liberación de objetos dinámicos.

Anexo 1

Cuadro 4: Tiempos de ejecución acumulados en microsegundos para *SinglyLinkedList*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	198,12
100	198,38	0	606,34	0	0	0	0	0	0	0
1000	1191,86	1785,58	4865,86	2106,16	0	0	1907,46	205,78	1198,98	0
10000	79296,66	213838,92	707527,56	190888,48	201,02	0	408650,92	0	80684,82	0
100000	11224738,88	23328046,46	177669376,50	22223775,00	199,74	397,6	182555531,50	1864,44	10885833,46	0

Cuadro 5: Tiempos de ejecución acumulados en microsegundos para *SinglyLinkedListTail*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	0
100	0	0	604,58	0	0	0	0	196,18	0	198,60
1000	1202,40	1800,46	4260,52	2029,00	0	0	1672,56	201,76	0	0
10000	74065,18	199575,74	1007083,72	220715,56	0	0	613371,48	0	0	304,34
100000	12179901,54	24430254,82	215502664,10	19816329,36	0	399,28	201797907,90	996,86	2440,22	2744,80

Cuadro 6: Tiempos de ejecución acumulados en microsegundos para *DoublyLinkedList*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	401,74
100	0	201,78	202,74	401,30	0	0	0	0	0	0
1000	1108,28	1782,68	4380,80	2012,54	0	0	1607,18	0	1074,52	0
10000	79185,38	213637,04	680926,40	185912,46	0	0	400135,52	198,52	82722,22	605,96
100000	13420073,06	27041040,30	180415732,70	23542248,60	302,96	253,4	333001923,60	1406,00	11993612,32	2644,84

Cuadro 7: Tiempos de ejecución acumulados en microsegundos para *DoublyLinkedListTail*

(N)	addAfter	addBefore	erase	find	getSize	isEmpty	popBack	popFront	pushBack	pushFront
10	0	0	0	0	0	0	0	0	0	0
100	396,90	0	796,86	0	0	0	0	0	0	0
1000	1179,30	2217,48	4399,88	1905,54	0	0	0	0	0	0
10000	72807,56	181413,70	679629,88	161699,52	0	0	200,82	402,28	203,24	0
100000	12720363,56	27044629,18	168086631,00	22484616,38	0	479,18	2608,62	4954,86	3173,26	7146,72

Cuadro 8: Tiempos de ejecución acumulados en microsegundos para *DynamicArrayStack*

(N)	delete	getSize	isEmpty	peek	pop	push
10	0	0	0	0	0	0
100	404,38	0	0	0	0	0
1000	2916,20	0	0	0	0	0
10000	158172,46	0	0	200,76	0	0
100000	28024617,44	460,1	661,76	1886,62	0	1696,52

Cuadro 9: Tiempos de ejecución acumulados en microsegundos para *DynamicArrayQueue*

(N)	delete	dequeue	enqueue	front	getSize	isEmpty
10	0	0	0	0	0	0
100	0	197,64	0	0	0	102,62
1000	3833,96	0	0	0	0	200,42
10000	236116,76	0	505,32	0	0	0
100000	41208081,14	903,9	6112,2	198,92	202,54	1255,92